# U-Net Segmentation Analysis

Comprehensive analysis of U-Net training on the E. coli growth stage detection dataset.

**Results Summary (Original Dataset):**

- Best mIoU (no background): **0.4013**
- Background IoU: 0.9134
- Rod IoU: 0.3955
- Dividing IoU: 0.3562
- Microcolony IoU: 0.3819

In [13]:
```
# Fix OpenMP error on Windows
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"

import json
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
import cv2
import torch
import sys

# Set style
try:
    plt.style.use('seaborn-v0_8-whitegrid')
except:
    try:
        plt.style.use('seaborn-whitegrid')
    except:
        pass  # Use default style

# Paths - use absolute paths
NOTEBOOK_DIR = Path(os.getcwd())
PROJECT_ROOT = Path(r"C:\Users\Alan\Codin\CS184A\E-coli-growth-stages-detection-
UNET_DIR = PROJECT_ROOT / "models" / "U-Net"
RESULTS_DIR = UNET_DIR / "results" / "original"
WEIGHTS_DIR = UNET_DIR / "weights"
DATA_DIR = PROJECT_ROOT / "data" / "unet_original"
CNN_RESULTS_DIR = PROJECT_ROOT / "models" / "CNN-sliding-window" / "results"

# Add src to path
sys.path.insert(0, str(UNET_DIR / "src"))

print(f"Project root: {PROJECT_ROOT}")
print(f"Results dir: {RESULTS_DIR}")
print(f"Data dir: {DATA_DIR}")
print(f"Results dir exists: {RESULTS_DIR.exists()}")
```

```
Project root: C:\Users\Alan\Codin\CS184A\E-coli-growth-stages-detection-and-class
ification
Results dir: C:\Users\Alan\Codin\CS184A\E-coli-growth-stages-detection-and-classi
fication\models\U-Net\results\original
Data dir: C:\Users\Alan\Codin\CS184A\E-coli-growth-stages-detection-and-classific
ation\data\unet_original
Results dir exists: True
```

# 1. Load Training History

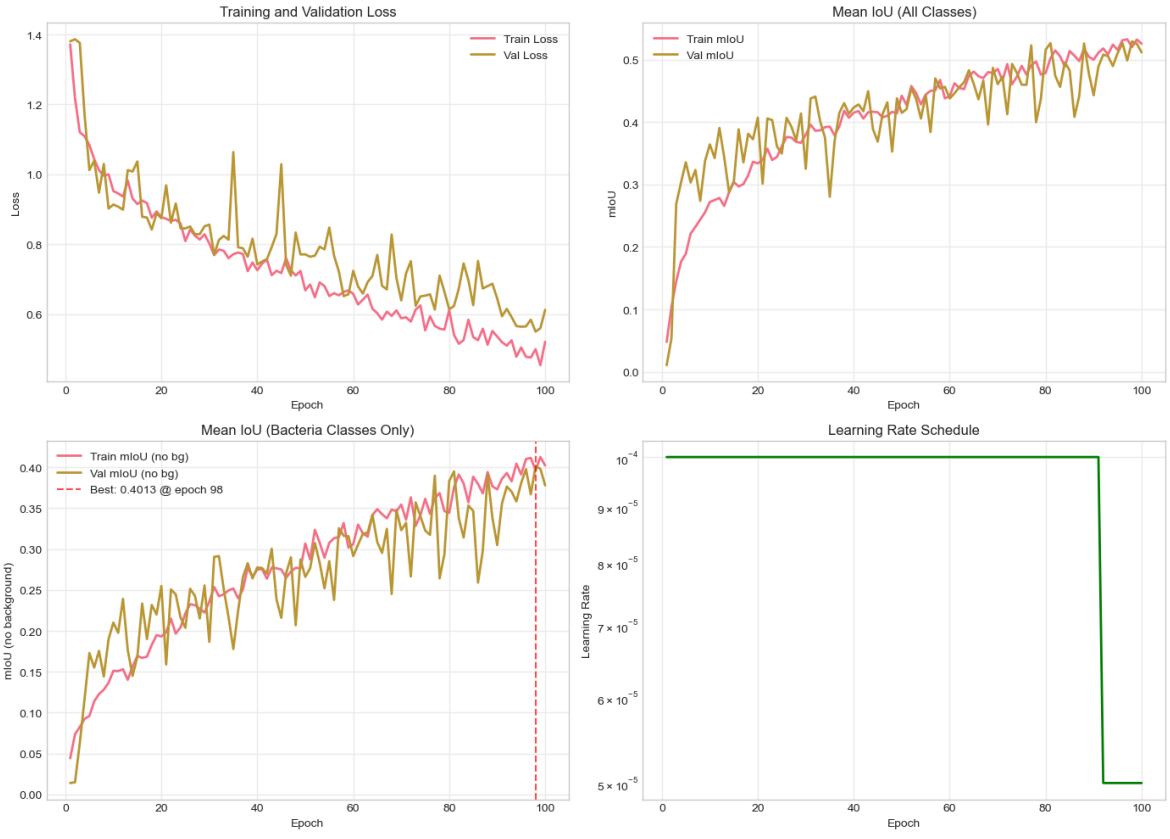
In [14]:
```python
# Load training history
history_path = RESULTS_DIR / "training_history.json"
print(f"Loading history from: {history_path}")
print(f"File exists: {history_path.exists()}")

with open(history_path, "r") as f:
    history = json.load(f)

epochs = list(range(1, len(history['train_loss']) + 1))

print(f"\nTraining completed: {len(epochs)} epochs")
print(f"Best Val mIoU (no bg): {max(history['val_miou_no_bg']):.4f}")
print(f"Best epoch: {np.argmax(history['val_miou_no_bg']) + 1}")
```

```
Loading history from: C:\Users\Alan\Codin\CS184A\E-coli-growth-stages-detection-a
nd-classification\models\U-Net\results\original\training_history.json
File exists: True

Training completed: 100 epochs
Best Val mIoU (no bg): 0.4013
Best epoch: 98
```

# 2. Training Curves

In [15]:
```python
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Loss curves
ax = axes[0, 0]
ax.plot(epochs, history['train_loss'], label='Train Loss', linewidth=2)
ax.plot(epochs, history['val_loss'], label='Val Loss', linewidth=2)
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss')
ax.set_title('Training and Validation Loss')
ax.legend()
ax.grid(True, alpha=0.3)

# mIoU curves
ax = axes[0, 1]
ax.plot(epochs, history['train_miou'], label='Train mIoU', linewidth=2)
ax.plot(epochs, history['val_miou'], label='Val mIoU', linewidth=2)
ax.set_xlabel('Epoch')
ax.set_ylabel('mIoU')
ax.set_title('Mean IoU (All Classes)')
ax.legend()
ax.grid(True, alpha=0.3)

# mIoU without background
ax = axes[1, 0]
```

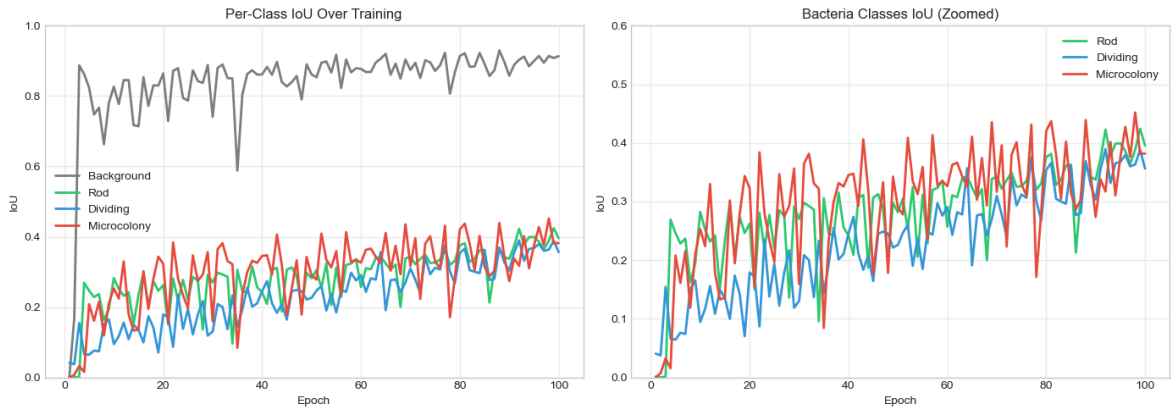
```
ax.plot(epochs, history['train_miou_no_bg'], label='Train mIoU (no bg)', linewid
ax.plot(epochs, history['val_miou_no_bg'], label='Val mIoU (no bg)', linewidth=2
best_epoch = int(np.argmax(history['val_miou_no_bg']) + 1)
best_miou = max(history['val_miou_no_bg'])
ax.axvline(x=best_epoch, color='red', linestyle='--', alpha=0.7, label=f'Best: {
ax.set_xlabel('Epoch')
ax.set_ylabel('mIoU (no background)')
ax.set_title('Mean IoU (Bacteria Classes Only)')
ax.legend()
ax.grid(True, alpha=0.3)

# Learning rate
ax = axes[1, 1]
ax.plot(epochs, history['lr'], linewidth=2, color='green')
ax.set_xlabel('Epoch')
ax.set_ylabel('Learning Rate')
ax.set_title('Learning Rate Schedule')
ax.set_yscale('log')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(RESULTS_DIR / 'training_curves.png', dpi=150, bbox_inches='tight')
plt.show()
```




# 3. Per-Class IoU Progression

In [16]:
```
# Extract per-class IoU over epochs
class_names = ['background', 'rod', 'dividing', 'microcolony']
colors = ['#808080', '#2ecc71', '#3498db', '#e74c3c']

iou_history = {name: [] for name in class_names}
for epoch_data in history['val_iou_per_class']:
    for name in class_names:
```

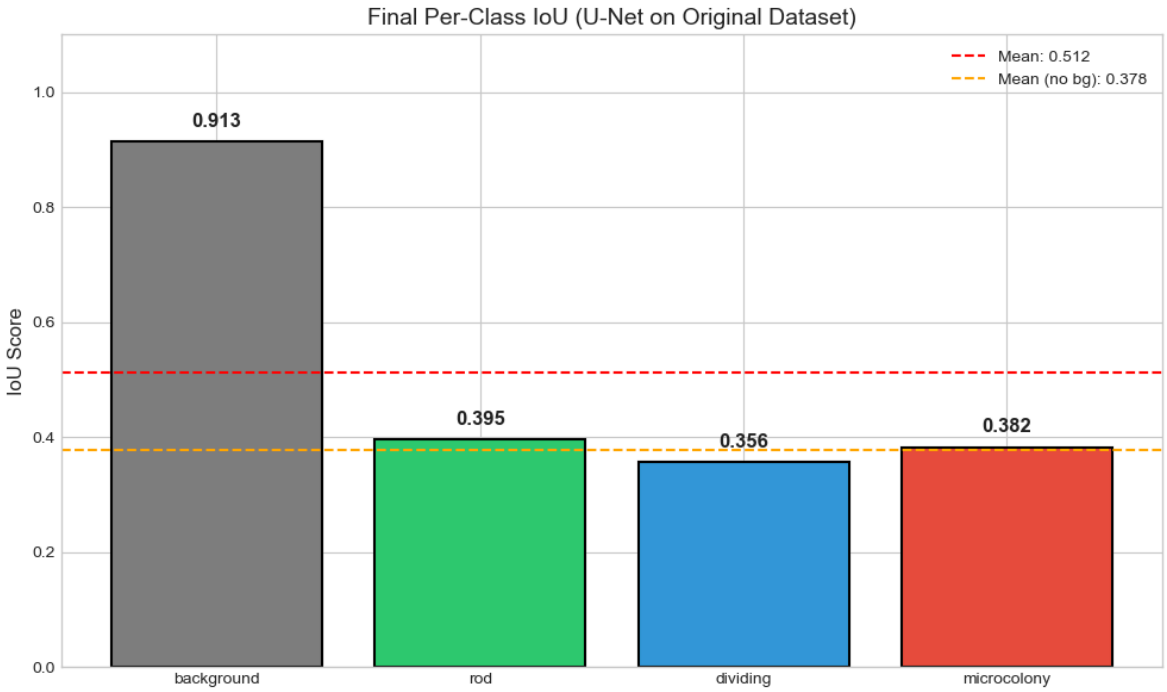
```
        val = epoch_data.get(name, None)
        iou_history[name].append(val if val is not None else np.nan)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# All classes
ax = axes[0]
for name, color in zip(class_names, colors):
    ax.plot(epochs, iou_history[name], label=name.capitalize(), linewidth=2, col
ax.set_xlabel('Epoch')
ax.set_ylabel('IoU')
ax.set_title('Per-Class IoU Over Training')
ax.legend()
ax.grid(True, alpha=0.3)
ax.set_ylim(0, 1)

# Bacteria classes only (zoomed)
ax = axes[1]
for name, color in zip(class_names[1:], colors[1:]):
    ax.plot(epochs, iou_history[name], label=name.capitalize(), linewidth=2, col
ax.set_xlabel('Epoch')
ax.set_ylabel('IoU')
ax.set_title('Bacteria Classes IoU (Zoomed)')
ax.legend()
ax.grid(True, alpha=0.3)
ax.set_ylim(0, 0.6)

plt.tight_layout()
plt.savefig(RESULTS_DIR / 'per_class_iou.png', dpi=150, bbox_inches='tight')
plt.show()
```




# 4. Final Performance Summary

In [17]:
```
# Get final IoU values
final_iou = history['val_iou_per_class'][-1]

fig, ax = plt.subplots(figsize=(10, 6))

classes = list(final_iou.keys())
ious = [final_iou[c] if final_iou[c] is not None else 0 for c in classes]

bars = ax.bar(classes, ious, color=colors, edgecolor='black', linewidth=1.5)

# Add value labels
for bar, iou in zip(bars, ious):
```

```
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height + 0.02,
            f'{iou:.3f}', ha='center', va='bottom', fontsize=12, fontweight='bol

ax.set_ylabel('IoU Score', fontsize=12)
ax.set_title('Final Per-Class IoU (U-Net on Original Dataset)', fontsize=14)
ax.set_ylim(0, 1.1)
ax.axhline(y=np.mean(ious), color='red', linestyle='--', label=f'Mean: {np.mean(
ax.axhline(y=np.mean(ious[1:]), color='orange', linestyle='--', label=f'Mean (no
ax.legend()

plt.tight_layout()
plt.savefig(RESULTS_DIR / 'final_iou_bar.png', dpi=150, bbox_inches='tight')
plt.show()

# Print summary
print("="*50)
print("FINAL PERFORMANCE SUMMARY")
print("="*50)
for name, iou in final_iou.items():
    iou_val = iou if iou is not None else 0
    print(f"{name:15}: {iou_val:.4f}")
print("-"*50)
print(f"{'Mean IoU':15}: {np.mean(ious):.4f}")
print(f"{'Mean IoU (no bg)':15}: {np.mean(ious[1:]):.4f}")
```




```
==================================================
FINAL PERFORMANCE SUMMARY
==================================================
background     : 0.9134
rod            : 0.3955
dividing       : 0.3562
microcolony    : 0.3819
--------------------------------------------------
Mean IoU       : 0.5117
Mean IoU (no bg): 0.3778
```

# 5. Load Model and Visualize Predictions

In [18]:
```
from models import UNetSmall

# Load model
device = torch.device('cpu')
model = UNetSmall(n_channels=1, n_classes=4).to(device)

model_path = WEIGHTS_DIR / "best_unet_original.pth"
print(f"Loading model from: {model_path}")

# PyTorch 2.6+ changed weights_only default to True
# Since we trust our own checkpoint, set weights_only=False
checkpoint = torch.load(model_path, map_location=device, weights_only=False)
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()

print(f"Model loaded from epoch {checkpoint['epoch'] + 1}")
print(f"Best metrics: {checkpoint['metrics']}")
```

```
Loading model from: C:\Users\Alan\Codin\CS184A\E-coli-growth-stages-detection-and
-classification\models\U-Net\weights\best_unet_original.pth
Model loaded from epoch 98
Best metrics: {'mean_iou': 0.5294389391396148, 'mean_iou_no_bg': 0.40125156836996
867}
```

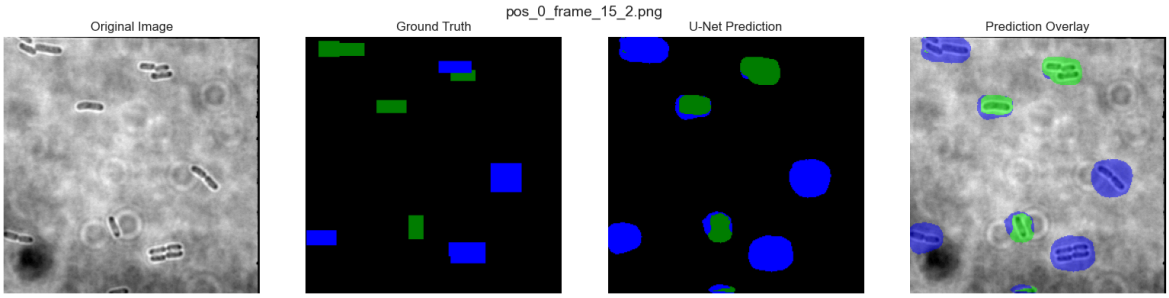
In [19]:
```
def predict_image(model, image_path, device):
    """Run inference on a single image."""
    # Load image
    image = cv2.imread(str(image_path), cv2.IMREAD_GRAYSCALE)

    # Preprocess
    img_tensor = torch.from_numpy(image).float().unsqueeze(0).unsqueeze(0) / 255
    img_tensor = (img_tensor - 0.5) / 0.5
    img_tensor = img_tensor.to(device)

    # Predict
    with torch.no_grad():
        output = model(img_tensor)
        pred = output.argmax(dim=1).squeeze().cpu().numpy()

    return image, pred

def visualize_prediction(image, mask_gt, mask_pred, title=""):
    """Visualize image, ground truth mask, and prediction."""
    fig, axes = plt.subplots(1, 4, figsize=(16, 4))

    # Color map for classes
    cmap = plt.cm.colors.ListedColormap(['black', 'green', 'blue', 'red'])
    bounds = [-0.5, 0.5, 1.5, 2.5, 3.5]
    norm = plt.cm.colors.BoundaryNorm(bounds, cmap.N)

    # Original image
    axes[0].imshow(image, cmap='gray')
    axes[0].set_title('Original Image')
    axes[0].axis('off')

    # Ground truth
    axes[1].imshow(mask_gt, cmap=cmap, norm=norm)
    axes[1].set_title('Ground Truth')
    axes[1].axis('off')
```

```
    # Prediction
    axes[2].imshow(mask_pred, cmap=cmap, norm=norm)
    axes[2].set_title('U-Net Prediction')
    axes[2].axis('off')

    # Overlay
    overlay = image.copy()
    overlay = cv2.cvtColor(overlay, cv2.COLOR_GRAY2RGB)

    # Add colored overlay for predictions
    colors_rgb = [(0, 0, 0), (0, 255, 0), (0, 0, 255), (255, 0, 0)]  # bg, rod,
    for cls_id in range(1, 4):
        mask = (mask_pred == cls_id)
        overlay[mask] = (overlay[mask] * 0.5 + np.array(colors_rgb[cls_id]) * 0.

    axes[3].imshow(overlay)
    axes[3].set_title('Prediction Overlay')
    axes[3].axis('off')

    plt.suptitle(title, fontsize=14)
    plt.tight_layout()
    return fig
```

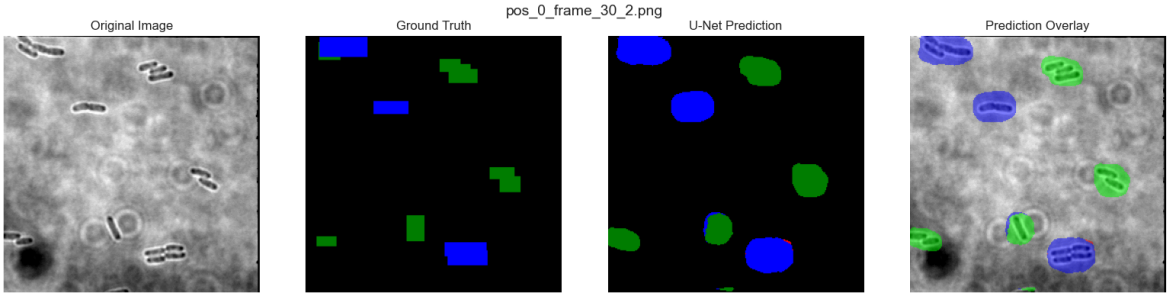
In [20]:

```
# Visualize predictions on validation set
val_images_dir = DATA_DIR / "val" / "images"
print(f"Looking for images in: {val_images_dir}")
print(f"Directory exists: {val_images_dir.exists()}")

val_images = sorted(val_images_dir.glob("*.png"))[:6]
print(f"Found {len(val_images)} images")

for img_path in val_images:
    image, pred = predict_image(model, img_path, device)

    # Load ground truth mask
    mask_path = DATA_DIR / "val" / "masks" / img_path.name
    mask_gt = cv2.imread(str(mask_path), cv2.IMREAD_GRAYSCALE)

    fig = visualize_prediction(image, mask_gt, pred, title=img_path.name)
    plt.show()
```

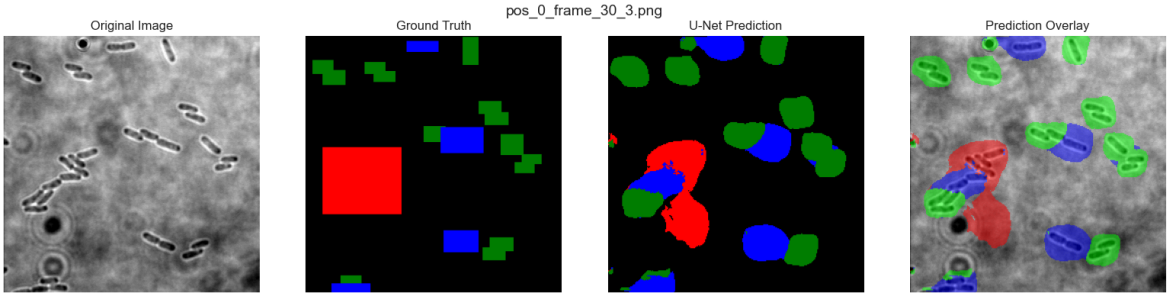
```
Looking for images in: C:\Users\Alan\Codin\CS184A\E-coli-growth-stages-detection-
and-classification\data\unet_original\val\images
Directory exists: True
Found 6 images
```



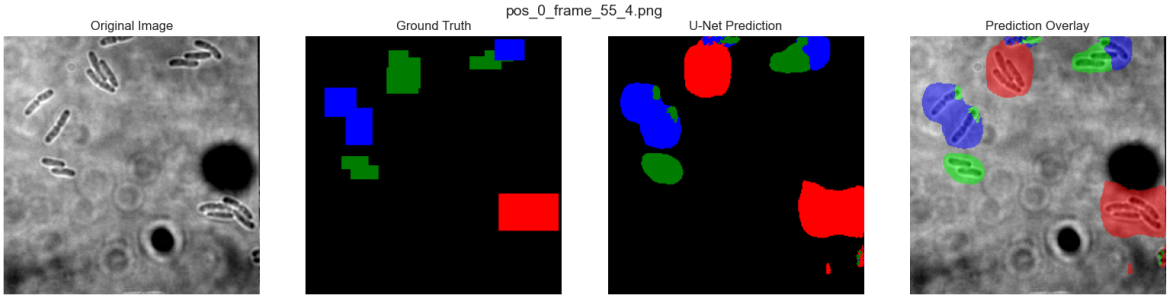
pos_0_frame_30_2.png



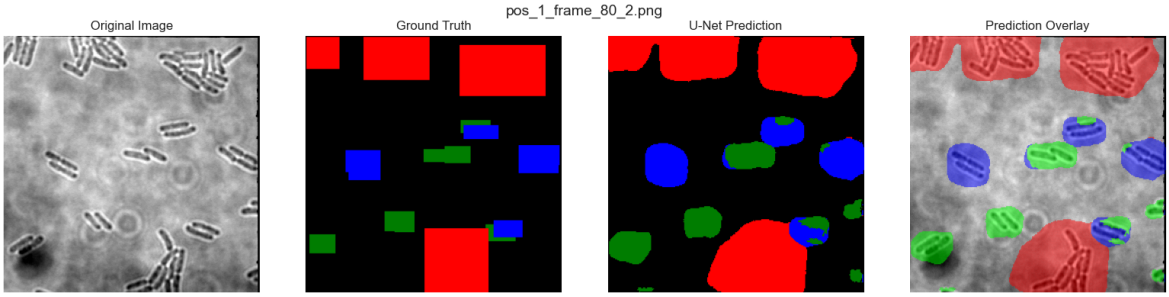

pos_0_frame_30_3.png




pos_0_frame_55_4.png




pos_1_frame_80_2.png




pos_2_frame_15_2.png